# CURSO DE GO
# GOLANG

Documentación Técnica Completa

*Prácticas y ejercicios cubiertos:*

Hello World · Truncar decimales · Find String · Slices
JSON · Leer archivos · Bubble Sort · Closures
Structs & Methods · Interfaces · Race Conditions
Parallel Sort · Dining Philosophers

# Introducción a Go (Golang)

Go es un lenguaje de programación compilado, estáticamente tipado y de código abierto desarrollado por Google. Está diseñado para ser simple, eficiente y confiable, con soporte nativo para concurrencia mediante goroutines y canales.

> **¿Por qué aprender Go?**
> Go combina la velocidad de lenguajes compilados como C con la simplicidad de lenguajes como Python. Es ideal para sistemas distribuidos, microservicios, herramientas de línea de comandos y aplicaciones con alta concurrencia.

## Conceptos Fundamentales

| Concepto | Descripción |
|---|---|
| `package main` | Declara el archivo como programa ejecutable (no librería) |
| `import` | Importa paquetes de la librería estándar o externos |
| `func main()` | Punto de entrada del programa; todo lo que contiene se ejecuta primero |
| `fmt` | Paquete para formateo e impresión en consola |
| `:=` | Declaración corta de variable con inferencia de tipo |
| `var` | Declaración explícita de variable |
| `go build` | Compila el código fuente y genera un ejecutable (.exe en Windows) |

# 1. Hello World

El programa más básico en Go: imprime texto en la consola. Sirve para entender la estructura mínima de cualquier programa Go.

## Estructura del programa

### Código fuente

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, World!")
}
```

### Explicación línea por línea

| Concepto | Descripción |
|---|---|
| `package main` | Indica que este archivo forma el paquete principal (ejecutable) |
| `import "fmt"` | Importa el paquete 'format' para imprimir en consola |
| `func main() { }` | Función de entrada. Go siempre comienza la ejecución aquí |
| `fmt.Println()` | Imprime texto seguido de un salto de línea |

## Compilación y ejecución

Para compilar y ejecutar en Windows (PowerShell):

```
go build HelloWorld.go    # Genera HelloWorld.exe
.\HelloWorld             # Ejecuta el programa
# Salida: Hello, World!
```

# 2. Truncar un Número Decimal

Programa que lee un número decimal desde la consola y lo trunca (convierte) al entero más cercano eliminando los decimales, sin redondear.

## Conceptos aplicados

- Declaración de variables con var
- Entrada de datos con fmt.Scan()
- Conversión de tipos: float64 → int

### Código fuente

```
package main

import "fmt"

func main() {
    var numero float64
    // 1. Leemos el número decimal del usuario
    fmt.Scan(&numero)
    // 2. Convertimos a int (esto trunca los decimales)
    resultado := int(numero)
    // 3. Imprimimos el resultado
    fmt.Println(resultado)
}
```

### Conceptos clave

| Concepto | Descripción |
|---|---|
| `float64` | Tipo de dato para números decimales de 64 bits |
| `int(numero)` | Conversión explícita de float64 a entero (trunca, no redondea) |
| `fmt.Scan(&var)` | Lee entrada del usuario; & pasa la dirección de memoria (puntero) |
| `&` | Operador de dirección: pasa la variable por referencia para que Scan pueda modificarla |

**Ejemplo de ejecución**

Entrada: 12.5 Salida: 12 Nota: int(12.9) = 12 (no redondea, solo elimina decimales)

# 3. Find String

Programa que solicita texto al usuario y verifica si contiene las tres letras "i", "a" y "n" (usando el paquete strings). Imprime "¡Encontrado!" si las tres están presentes, o "¡No encontrado!" si falta alguna.

## Código fuente

```
package main

import (
    "fmt"
    "strings"
)

func main() {
    var texto string
    fmt.Scan(&texto)
    // Convertimos a minúsculas para búsqueda insensible a mayúsculas
    texto = strings.ToLower(texto)
    // Verificamos presencia de las tres letras simultáneamente
    if strings.Contains(texto, "i") && strings.Contains(texto, "a") &&
strings.Contains(texto, "n") {
        fmt.Println("¡Encontrado!")
    } else {
        fmt.Println("¡No encontrado!")
    }
}
```

## Funciones del paquete strings

| Concepto | Descripción |
|---|---|
| `strings.ToLower()` | Convierte toda la cadena a minúsculas para comparación uniforme |
| `strings.Contains(s, sub)` | Retorna true si la cadena s contiene la subcadena sub |
| `&& (AND lógico)` | Evalúa múltiples condiciones; todas deben ser verdaderas |

**Ejemplo de ejecución**

Entrada: gian → ¡Encontrado! (contiene i, a y n) Entrada: dana → ¡No encontrado! (no contiene i)

# 4. Slices (Rebanadas)

Los Slices en Go son estructuras de datos dinámicas similares a los arrays en otros lenguajes, pero con tamaño variable. Este programa captura números enteros del usuario, los almacena en un slice y los ordena automáticamente con cada inserción.

## ¿Qué es un Slice?

Un slice es una referencia a un segmento de un array subyacente. A diferencia de los arrays (tamaño fijo), los slices pueden crecer dinámicamente usando la función append().

### Código fuente

```go
package main

import (
    "fmt"
    "sort"
    "strconv"
)

func main() {
    rebanada := make([]int, 0)  // Slice vacío de enteros
    for {
        var entrada string
        fmt.Scan(&entrada)
        // Intentamos convertir la entrada a entero
        numero, err := strconv.Atoi(entrada)
        if err != nil {
            break  // Si no es número, terminamos
        }
        rebanada = append(rebanada, numero)  // Agregamos al slice
        sort.Ints(rebanada)                   // Ordenamos de menor a mayor
        fmt.Println(rebanada)                 // Mostramos estado actual
    }
}
```

### Funciones y conceptos clave

| Concepto | Descripción |
|---|---|
| `make([]int, 0)` | Crea un slice vacío de enteros con capacidad inicial 0 |
| `append(slice, elem)` | Agrega un elemento al slice y retorna el slice modificado |
| `sort.Ints(slice)` | Ordena un slice de enteros in-place de menor a mayor |
| `strconv.Atoi(str)` | Convierte string a int; retorna (int, error) |

| | |
|---|---|
| `err != nil` | Comprueba si ocurrió un error en la conversión |
| `for { }` | Bucle infinito; se rompe con break |

# 5. JSON de Ordenamiento

Programa que solicita nombre y dirección al usuario, crea una estructura Go (struct) con esos datos y la serializa a formato JSON usando el paquete encoding/json.

## Código fuente

```
package main

import (
    "encoding/json"
    "fmt"
)

// Definimos la estructura del objeto con etiquetas JSON
type Persona struct {
    Nombre   string `json:"nombre"`
    Direccion string `json:"direccion"`
}

func main() {
    var n, d string
    fmt.Scan(&n)
    fmt.Scan(&d)
    // Creamos instancia de la estructura con los datos
    p := Persona{Nombre: n, Direccion: d}
    // Serializamos a JSON (Marshalling)
    datosJSON, err := json.Marshal(p)
    if err != nil { return }
    fmt.Println(string(datosJSON))
}
```

## Conceptos clave

| Concepto | Descripción |
|---|---|
| `struct` | Tipo de dato compuesto que agrupa campos con nombre y tipo |
| `` `json:"nombre"` `` | Etiqueta que indica el nombre de la clave en el JSON resultante |
| `json.Marshal(v)` | Serializa una estructura Go a bytes JSON (Marshalling) |
| `string(bytes)` | Convierte []byte a string para imprimir |

### Ejemplo de ejecución

Entrada: gian Entrada: tlaxcoapan Salida: {"nombre":"gian","direccion":"tlaxcoapan"} Nota: fmt.Scan() no captura espacios. Para texto con espacios usar bufio.Scanner.

# 6. Leer Archivo y Almacenar en Estructuras

Programa que abre un archivo de texto con nombres y apellidos (uno por línea, separados por espacio), lee cada línea, separa los campos y los guarda en un slice de structs con campos individuales (Fname, Lname).

## Código fuente

```go
package main

import (
    "bufio"
    "fmt"
    "os"
    "strings"
)

type Nombre struct {
    Fname string
    Lname string
}

func main() {
    var nombreArchivo string
    fmt.Print("Introduce el nombre del archivo: ")
    fmt.Scan(&nombreArchivo)

    archivo, err := os.Open(nombreArchivo)
    if err != nil {
        fmt.Println("Error al abrir:", err)
        return
    }
    defer archivo.Close()

    personas := make([]Nombre, 0)
    scanner := bufio.NewScanner(archivo)
    for scanner.Scan() {
        partes := strings.Split(scanner.Text(), " ")
        if len(partes) >= 2 {
            p := Nombre{Fname: partes[0], Lname: partes[1]}
            personas = append(personas, p)
        }
    }
    for _, p := range personas {
        fmt.Printf("Nombre: %s, Apellido: %s\n", p.Fname, p.Lname)
    }
}
```

## Funciones de manejo de archivos

| Concepto | Descripción |
|---|---|
| `os.Open(nombre)` | Abre un archivo; retorna (*File, error) |
| `defer archivo.Close()` | Cierra el archivo al terminar la función (garantizado) |
| `bufio.NewScanner(f)` | Crea un lector eficiente línea por línea |
| `scanner.Scan()` | Avanza al siguiente token/línea; retorna false al terminar |
| `scanner.Text()` | Retorna el texto de la línea actual como string |
| `strings.Split(s, " ")` | Divide un string por el separador y retorna []string |

# 7. Bubble Sort

Implementación del algoritmo de ordenamiento Burbuja. El usuario ingresa hasta 10 números enteros, y el programa los ordena de menor a mayor usando dos funciones separadas: Swap() y BubbleSort().

## ¿Cómo funciona Bubble Sort?

El algoritmo compara pares de elementos adyacentes y los intercambia si están en orden incorrecto. Repite este proceso varias pasadas hasta que el arreglo queda ordenado. Complejidad: $O(n^2)$ en el peor caso.

### Código fuente

```go
package main

import "fmt"

// Swap intercambia dos elementos adyacentes en el slice
func Swap(numbers []int, i int) {
    numbers[i], numbers[i+1] = numbers[i+1], numbers[i]
}

// BubbleSort ordena el slice in-place de menor a mayor
func BubbleSort(numbers []int) {
    n := len(numbers)
    for i := 0; i < n; i++ {
        for j := 0; j < n-i-1; j++ {
            if numbers[j] > numbers[j+1] {
                Swap(numbers, j)
            }
        }
    }
}

func main() {
    var input int
    numbers := make([]int, 0, 10)
    fmt.Println("Ingrese hasta 10 números (no numérico para terminar):")
    for i := 0; i < 10; i++ {
        fmt.Printf("Número %d: ", i+1)
        _, err := fmt.Scan(&input)
        if err != nil { break }
        numbers = append(numbers, input)
    }
    fmt.Println("\nLista original:", numbers)
    BubbleSort(numbers)
    fmt.Println("Lista ordenada:", numbers)
```

```
}
```

### Intercambio simultáneo en Go

Go permite intercambiar dos valores en una sola línea: a, b = b, a Esto es equivalente a usar una variable temporal en otros lenguajes, pero es más conciso y legible.

# 8. GenDisplaceFn — Clausuras (Closures)

Programa que utiliza clausuras (funciones como valores) para crear una fábrica de fórmulas físicas. El usuario proporciona aceleración, velocidad inicial y desplazamiento inicial; el sistema genera una función personalizada que recuerda esos datos y calcula el desplazamiento para cualquier tiempo t.

## ¿Qué es una Clausura (Closure)?

Una clausura es una función que captura (recuerda) las variables del entorno donde fue creada. En Go, las funciones son ciudadanos de primera clase: pueden pasarse como argumentos y retornarse desde otras funciones.

### Fórmula cinemática implementada

```
s = (0.5 × a × t²) + (vo × t) + so

Donde:
  s  = desplazamiento final
  a  = aceleración
  vo = velocidad inicial
  so = desplazamiento inicial
  t  = tiempo
```

### Código fuente

```go
package main

import (
    "fmt"
    "math"
)

// GenDisplaceFn retorna una clausura que recuerda a, vo y so
func GenDisplaceFn(a, vo, so float64) func(float64) float64 {
    return func(t float64) float64 {
        return (0.5 * a * math.Pow(t, 2)) + (vo * t) + so
    }
}

func main() {
    var a, vo, so, t float64
    fmt.Scan(&a)
    fmt.Scan(&vo)
    fmt.Scan(&so)
    fn := GenDisplaceFn(a, vo, so)  // Genera la función personalizada
    fmt.Scan(&t)
```

```
    fmt.Printf("Desplazamiento: %.2f\n", fn(t))
}
```

**Ejemplo de ejecución**

a=10, vo=2, so=1, t=3 Resultado: (0.5×10×9) + (2×3) + 1 = 45 + 6 + 1 = 52.00

# 9. Animals — Structs y Métodos

Programa que organiza información de animales (vaca, pájaro, serpiente) usando structs y métodos. Define tres métodos (Eat, Move, Speak) sobre el struct Animal y permite consultar propiedades via un menú de comandos interactivo.

## Structs y Métodos en Go

En Go no hay clases. Se usan structs para agrupar datos y métodos para definir comportamiento. Un método es una función con un receptor (receiver) de un tipo específico.

### Código fuente

```
type Animal struct {
    food       string
    locomotion string
    noise      string
}

// Método con receptor de tipo Animal
func (a Animal) Eat()   { fmt.Println(a.food) }
func (a Animal) Move()  { fmt.Println(a.locomotion) }
func (a Animal) Speak() { fmt.Println(a.noise) }

func main() {
    animals := map[string]Animal{
        "vaca":      {food: "hierba", locomotion: "caminar", noise: "mugido"},
        "pájaro":    {food: "gusanos", locomotion: "volar",   noise: "pío"},
        "serpiente": {food: "ratones", locomotion: "deslizarse", noise:
"hsss"},
    }
    // Bucle interactivo para consultar animales...
}
```

### Tabla de datos de animales

| Concepto | Descripción |
|---|---|
| `vaca` | Come hierba, camina, hace mugido |
| `pájaro` | Come gusanos, vuela, hace pío |
| `serpiente` | Come ratones, se desliza, hace hsss |

# 10. Animal Interface Factory

Evolución del programa anterior: ahora usa interfaces para lograr polimorfismo. Permite crear instancias de animales con nombre personalizado (newanimal) y consultarlas (query), demostrando el patrón de fábrica en Go.

## Interfaces en Go

Una interfaz define un conjunto de métodos que un tipo debe implementar. La implementación es implícita: si un tipo tiene todos los métodos de la interfaz, la cumple automáticamente (duck typing).

### Definición de la interfaz y tipos

```
// Interfaz: define el contrato
type Animal interface {
    Eat()
    Move()
    Speak()
}

// Cada struct implementa la interfaz automáticamente
type Cow   struct{}
type Bird  struct{}
type Snake struct{}

func (c Cow)   Eat()   { fmt.Println("hierba") }
func (c Cow)   Move()  { fmt.Println("caminar") }
func (c Cow)   Speak() { fmt.Println("mugido") }
// ... (misma estructura para Bird y Snake)
```

### Motor de comandos

```
animalMap := make(map[string]Animal)

// Comando: newanimal <nombre> <tipo>
// Ejemplo: newanimal firulais vaca
animalMap[name] = Cow{}

// Comando: query <nombre> <accion>
// Ejemplo: query firulais comer
ani.Eat()   // comer
ani.Move()  // moverse
ani.Speak() // hablar
```

**Diferencia: Struct vs Interface**

> Struct: define datos y métodos concretos para un tipo específico. Interface: define solo la firma de métodos, sin implementación. Permite polimorfismo: una variable de tipo interfaz puede apuntar a cualquier struct que la implemente.

# 11. Race Condition — Condición de Carrera

Demostración didáctica de un error clásico en programación concurrente: la condición de carrera. El programa lanza dos goroutines que incrementan simultáneamente la misma variable compartida, mostrando que el resultado es impredecible.

## ¿Qué es una Race Condition?

Es un fallo que ocurre cuando el resultado de un programa depende del orden impredecible de ejecución de dos o más goroutines que acceden y modifican un recurso compartido al mismo tiempo.

## ¿Por qué ocurre?

Incrementar una variable no es una operación atómica. Requiere tres pasos:

- LEER el valor actual de la memoria
- SUMAR uno al valor leído
- ESCRIBIR el resultado de vuelta en memoria

Si dos goroutines ejecutan estos pasos intercaladas, una puede sobrescribir el trabajo de la otra.

### Código fuente

```
package main

import (
    "fmt"
    "time"
)

func main() {
    variableCompartida := 0

    // Goroutine 1: incrementa 100 veces
    go func() {
        for i := 0; i < 100; i++ {
            valor := variableCompartida  // 1. LEE
            time.Sleep(time.Millisecond) // Fuerza interrupción
            variableCompartida = valor + 1 // 2. ESCRIBE
        }
    }()

    // Goroutine 2: misma operación concurrente
    go func() {
        for i := 0; i < 100; i++ {
```

```
            valor := variableCompartida
            time.Sleep(time.Millisecond)
            variableCompartida = valor + 1
        }
    }()

    time.Sleep(2 * time.Second)  // Espera a que terminen
    // Esperado: 200. Real: número menor e impredecible
    fmt.Printf("Valor final: %d\n", variableCompartida)
}
```

### Solución: sync.Mutex

Para evitar race conditions se usa un Mutex (exclusión mutua): var mu sync.Mutex mu.Lock() // Bloquea el acceso variable++ // Operación segura mu.Unlock() // Libera el acceso Esto garantiza que solo una goroutine modifica la variable a la vez.

# 12. Parallel Sort Dispatch

Programa avanzado de concurrencia: divide un array en 4 particiones y ordena cada partición en paralelo usando goroutines independientes. Usa sync.WaitGroup para esperar que todas terminen y luego fusiona los resultados.

## Arquitectura del programa

- Usuario ingresa N números enteros
- El array se divide en 4 submatrices de tamaño (N+3)/4
- Se lanzan 4 goroutines, cada una ordena su partición
- sync.WaitGroup espera a que todas las goroutines terminen
- La goroutine principal fusiona y ordena la lista completa

### Código fuente

```
package main

import (
    "fmt"
    "sort"
    "sync"
)

func sortSubarray(arr []int, wg *sync.WaitGroup) {
    defer wg.Done()  // Marca la goroutine como completada al terminar
    fmt.Println("Goroutine ordenando submatriz:", arr)
    sort.Ints(arr)
}

func main() {
    // ... lectura de datos ...
    size := (len(slice) + 3) / 4
    var wg sync.WaitGroup

    for i := 0; i < 4; i++ {
        start := i * size
        end   := min((i+1)*size, len(slice))
        wg.Add(1)                              // Registra 1 goroutine
        go sortSubarray(slice[start:end], &wg) // Lanza goroutine
    }
    wg.Wait()          // Espera que todas terminen
    sort.Ints(slice)   // Fusión final
    fmt.Println(slice)
}
```

## sync.WaitGroup — API

| Concepto | Descripción |
|---|---|
| `wg.Add(n)` | Registra n goroutines que deben completarse |
| `wg.Done()` | Decrementa el contador (llamar con defer al inicio de cada goroutine) |
| `wg.Wait()` | Bloquea hasta que el contador llega a cero (todas terminaron) |

# 13. Dining Philosophers — Filósofos Comensales

Implementación del clásico problema de sincronización de Dijkstra. Cinco filósofos comparten cinco palillos y deben comer exactamente tres veces cada uno. Un anfitrión (goroutine) actúa como semáforo para evitar deadlocks.

## El Problema

5 filósofos en una mesa circular, con 1 palillo entre cada par. Para comer necesitan 2 palillos (izquierdo y derecho). Si todos toman su palillo izquierdo simultáneamente, ninguno puede tomar el derecho → DEADLOCK.

## Solución: Anfitrión como Semáforo

La goroutine anfitriona limita el número de filósofos comiendo simultáneamente a máximo 2, rompiendo el ciclo de espera circular.

### Estructuras principales

```
type ChopS struct{ sync.Mutex }   // Palillo = Mutex

type Philo struct {
    leftChop, rightChop *ChopS
    id int
}

// Goroutine anfitriona: tokens = capacidad máxima simultánea
func host(request chan bool, done chan bool) {
    tokens := 2  // Máximo 2 filósofos a la vez
    for {
        select {
        case <-request:
            for tokens == 0 { <-done; tokens++ }
            tokens--
        case <-done:
            tokens++
        }
    }
}
```

### Protocolo de cada filósofo

| Concepto | Descripción |
|---|---|
| `1. request <- true` | Pide permiso al anfitrión para comer |

| | |
|---|---|
| `2. leftChop.Lock()` | Toma el palillo izquierdo |
| `3. rightChop.Lock()` | Toma el palillo derecho |
| `4. fmt.Printf()` | Imprime 'empezando a comer N' |
| `5. time.Sleep(100ms)` | Simula el tiempo comiendo |
| `6. fmt.Printf()` | Imprime 'terminando de comer N' |
| `7. Unlock() ambos` | Devuelve los palillos |
| `8. done <- true` | Notifica al anfitrión que terminó |

### Canales en Go

Los canales (chan) son el mecanismo nativo de Go para comunicación entre goroutines. ch <- valor // Enviar al canal (bloquea si está lleno) valor = <-ch // Recibir del canal (bloquea si está vacío) Esta comunicación sincronizada elimina la necesidad de mutexes en muchos casos.

# Resumen de Conceptos Aprendidos

## Fundamentos del Lenguaje

| Concepto | Descripción |
|---|---|
| `package / import` | Estructura de paquetes y dependencias |
| `var / :=` | Declaración explícita e inferida de variables |
| `func` | Funciones con múltiples valores de retorno |
| `struct` | Tipos compuestos definidos por el usuario |
| `interface` | Polimorfismo implícito (duck typing) |
| `Métodos (receivers)` | Comportamiento asociado a tipos: func (t Type) Method() |

## Estructuras de Datos

| Concepto | Descripción |
|---|---|
| `Slice ([]T)` | Array dinámico; make(), append(), len(), cap() |
| `Map (map[K]V)` | Diccionario clave-valor; operaciones O(1) promedio |
| `Struct` | Agrupación de campos heterogéneos con nombre |

## Concurrencia

| Concepto | Descripción |
|---|---|
| `goroutine` | Hilo ligero: go func() {}() |
| `channel (chan)` | Comunicación segura entre goroutines |
| `sync.Mutex` | Exclusión mutua para proteger recursos compartidos |
| `sync.WaitGroup` | Sincronización: esperar N goroutines |
| `select` | Multiplexado de canales (como switch para canales) |
| `Race Condition` | Fallo por acceso concurrente no sincronizado |
| `Deadlock` | Bloqueo circular de goroutines esperándose mutuamente |

## Paquetes Estándar Utilizados

| Concepto | Descripción |
|---|---|
| `fmt` | Entrada/salida formateada (Println, Scanf, Printf) |
| `os` | Operaciones del sistema operativo (Open, Args) |
| `bufio` | Entrada/salida con buffer (Scanner, línea por línea) |
| `strings` | Manipulación de cadenas (Contains, ToLower, Split) |
| `strconv` | Conversión de tipos (Atoi, Itoa) |
| `sort` | Algoritmos de ordenamiento (Ints, Strings) |
| `encoding/json` | Serialización JSON (Marshal, Unmarshal) |
| `sync` | Primitivas de sincronización (Mutex, WaitGroup) |
| `math` | Funciones matemáticas (Pow, Sqrt, Abs) |
| `time` | Tiempo y duraciones (Sleep, Now, Duration) |